SYSTEM DESIGN · AS BUILT, 28 OCT 2026

RUBIIK ONE *store*

A pre-booking store for one limited edition of 200 numbered watches. A React single-page site talks to one Express API, which runs locally as a Node server and on Vercel as a serverless function, backed by MongoDB Atlas and Razorpay.

200 pieces, shared by 2 models | Automatic Skeleton **₹49,999** | Quartz Olive **₹39,999**

MRPs incl. **18% GST** | Pre-booking **−20%** | Launch **6 Nov 2026** | Delivery from **25 Dec 2026**

---

01 Overview

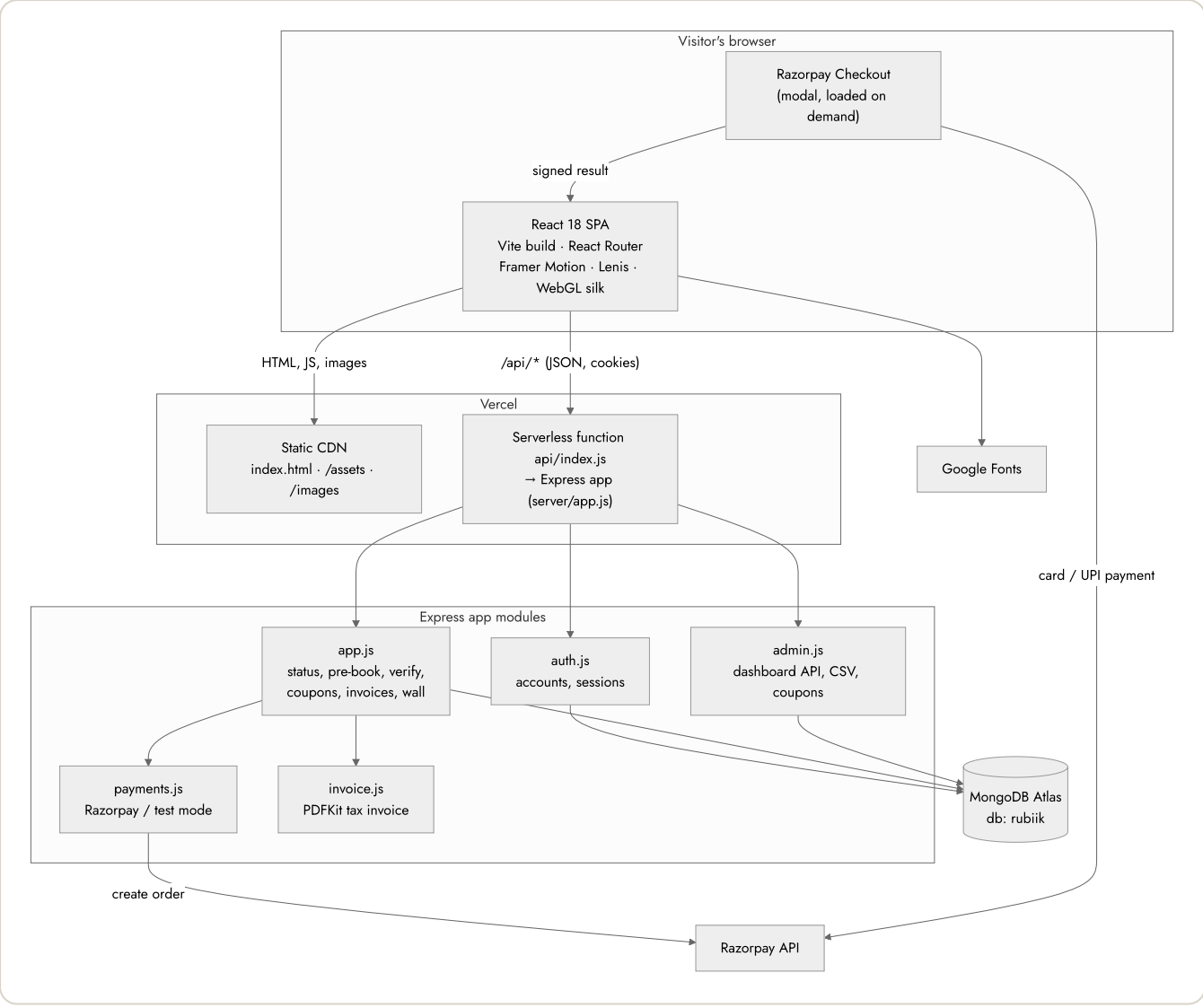
What the system does, end to end.

- **Storefront:** a motion-led home page (hero, Founding Owners wall, philosophy, collection, gallery, details, pre-booking), product pages, legal pages.
- **Accounts & checkout:** customers sign up, enter delivery details (with state, for GST), optionally apply an admin-issued coupon, and pay through Razorpay.
- **Edition:** on payment each watch gets the next edition number and a keyed serial (`RBK-AS-017-7F3C`) on a Certificate of Edition.
- **Paperwork:** each paid order gets a sequential GST tax invoice (`RBK/26-27/0001`) as a PDF.
- **Admin:** a password-protected dashboard for orders, customers, coupons, waitlist, the owners wall, media and the design system.

One module, `src/data/catalog.js` , holds prices, dates and every price calculation. Both the browser and the server import it, so displayed and charged amounts can't drift apart. The server still recomputes every amount and never trusts the client's numbers.

02 Architecture

Production on Vercel. Locally, Vite serves the site on :5173 and proxies `/api` to the same Express app on :8787.



The function caches its Express app and database connection while warm; a failed start returns 503 and the next request retries.

03 Technology

LAYER	CHOICE	WHY
UI	React 18.3, React Router 6.28, Vite 5.4	Single-page site with lazy-loaded routes and fast builds.
Styling	Tailwind CSS 3.4, custom tokens (night, ivory, wine, gold)	Theme colours #0B1219 / #A50003 / #F9F9F9; Cormorant Garamond + Jost.
Motion	Framer Motion 11, Lenis 1.1, WebGL shader, canvas particles	Scroll reveals, parallax, smooth scroll, the hero's silk background.
Server	Node + Express 4.21, compression	One app for local, self-hosted and Vercel.

LAYER	CHOICE	WHY
Database	MongoDB 6.20 driver → Atlas	Transactions keep edition and invoice numbering exact.
Payments	Razorpay Orders + Checkout; HMAC-SHA256 verification	Test mode without keys; test keys set locally.
Documents	PDFKit 0.20.2	Server-side GST invoice PDFs.
Images	sharp (build tool), WebP with 768 / 1152 px variants	Responsive `srcset` without losing quality.
Hosting	Vercel (static + one function), GitHub	Deploys on push to `main`.

04 Front end

Routes

PATH	PAGE	LOADING
`/`	Home: all sections	In the main bundle
`/watch/:slug`	Product page (front / caseback views, specs, pre-book)	Lazy
`/checkout`	Sign in → details → payment → certificate	Lazy
`/account`	Certificates, invoices, orders awaiting payment	Lazy
`/privacy`, `/terms`	Legal pages (company, GSTIN, contact from `legal.js`)	Lazy
`/admin/*`	Admin dashboard with its own sign-in and layout	Lazy

Home page, in order

Hero (Dhanteras theme until 7 Nov) → Founding Owners wall → marquee → I · Philosophy → II · Signature collection → III · Meaning → IV · Vision → Design gallery → Automatic blueprint poster → Details comparison → Experience → Quartz blueprint poster → Limited edition → IX · Pre-booking (launch date, countdown, timeline, product cards) → Finale → Footer.

Client state

ShopProvider `lib/shop.jsx`

Cart (in `localStorage`), sales phase and live stock from `/api/status` (refreshed every 30 s), totals via `totalsFor()`.

AuthProvider `lib/auth.jsx`

The signed-in customer from `/api/auth/me`; sign in, register, sign out.

Shared data `src/data/`

`catalog.js` (products, dates, pricing), `legal.js` (company, GSTIN, contact), `states.js` (GST state codes), `images.json` (responsive variants).

Sales phase

Decided by the server clock: **teaser** before 24 Oct, **prebook** until 6 Nov, **launched** after. Copy, prices and buttons follow it automatically.

05 Back end & API

28 routes. PUBLIC needs nothing, CUSTOMER needs the `rbk_session` cookie, ADMIN needs the `rbk_admin` cookie.

METHOD & PATH	ACCESS	PURPOSE
`GET /api/status`	PUBLIC	Phase, server time, launch settings, stock per model
`GET /api/owners`	PUBLIC	Founding Owners wall (opted-in display names only)
`POST /api/waitlist`	PUBLIC	"Notify me" email
`POST /api/coupons/check`	PUBLIC	Validate a coupon code (20 checks / 15 min per IP)
`POST /api/auth/register` · `login` · `logout`, `GET /api/auth/me`	PUBLIC	Customer accounts and sessions
`POST /api/prebook`	CUSTOMER	Validate details, recompute prices, apply coupon, reserve pieces for 30 min, start payment

METHOD & PATH	ACCESS	PURPOSE
`POST /api/orders/:id/verify`	CUSTOMER	Verify payment signature, issue edition numbers, serials and invoice number
`GET /api/orders/:id/invoice`	CUSTOMER	Own invoice PDF (`?download=1` to save)
`GET /api/me/orders`	CUSTOMER	Own paid and held orders
`POST /api/admin/login · logout` , `GET /api/admin/me`	PUBLIC	Admin sign-in (5 attempts / 15 min)
`GET /api/admin/overview · orders · customers · waitlist · assets`	ADMIN	Dashboard data
`GET /api/admin/orders.csv · waitlist.csv`	ADMIN	CSV exports (formula-injection safe)
`GET /api/admin/orders/:id/invoice`	ADMIN	Any order's invoice PDF
`PATCH /api/admin/orders/:id/wall`	ADMIN	Show / hide a card, edit its display name
`GET · POST /api/admin/coupons` , `PATCH · DELETE /api/admin/coupons/:code`	ADMIN	List, create, pause / resume, delete unused coupons

server/app.js

Builds the app, connects the database, mounts the other modules; owns stock, pre-booking, payment verification, coupons, invoices and the owners wall.

server/auth.js

scrypt password hashes, random session tokens stored as SHA-256 hashes, 30-day httpOnly cookie, login and register limits.

server/admin.js

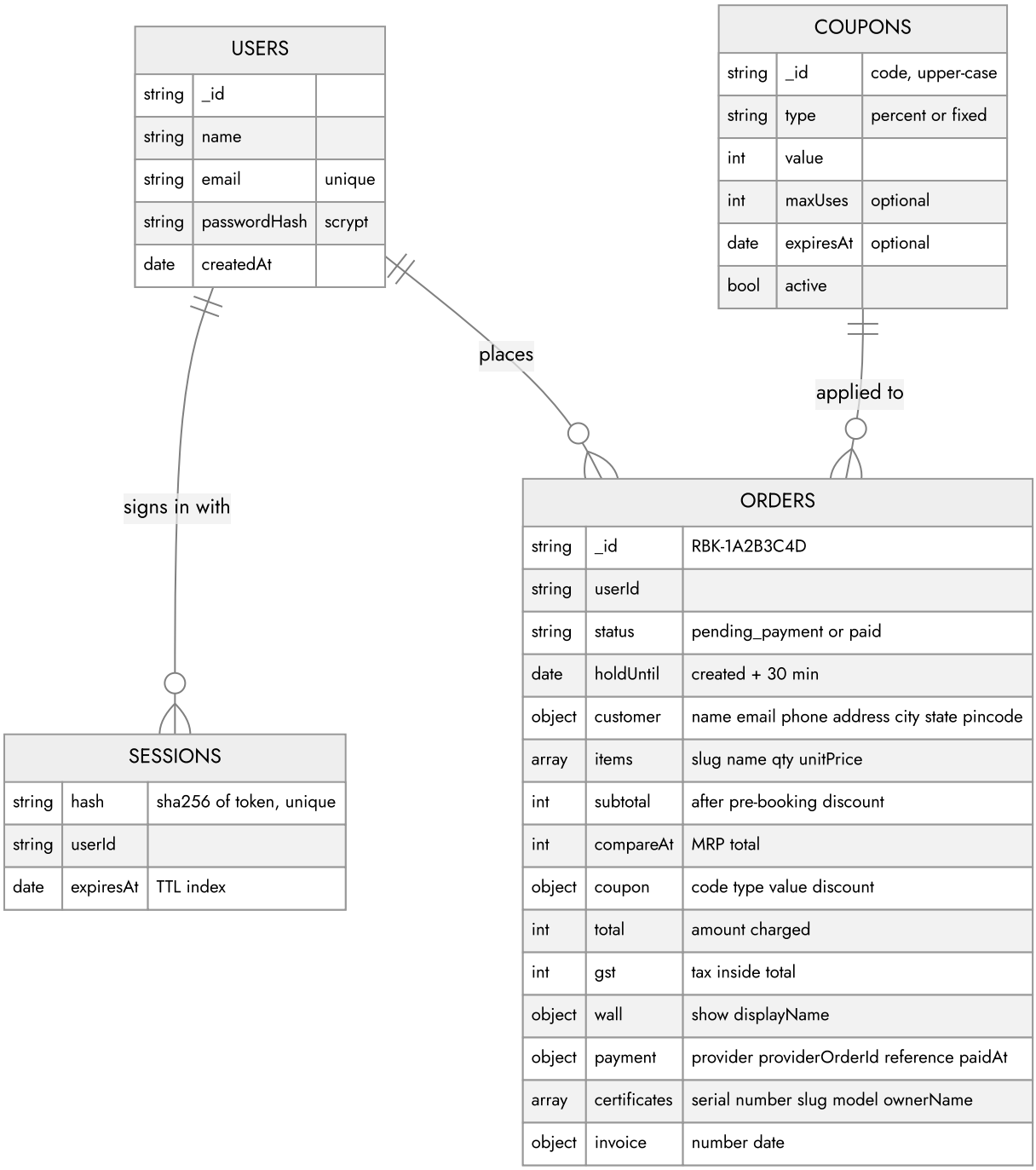
Admin login from env username/password, 12-hour session, every dashboard route, coupons, CSV.

server/payments.js · invoice.js · db.js

Razorpay order creation and signature checks; the PDF tax invoice; Mongo connection, collections, indexes, transactions.

06 Data model

MongoDB database `rubiik`. Amounts are whole rupees, GST-inclusive.



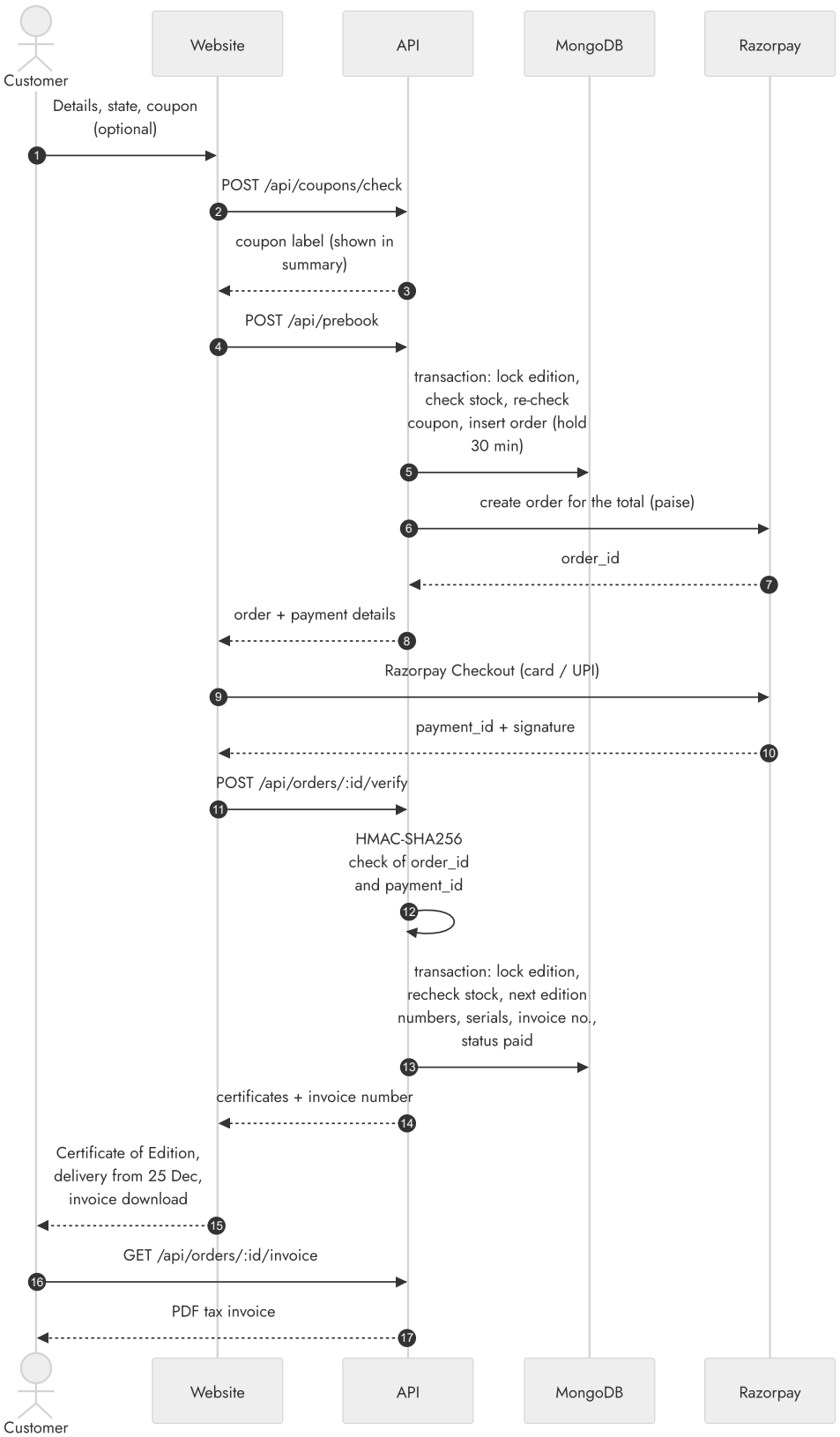
COLLECTION	FIELDS	USE
`waitlist`	email (unique), at	"Notify me" sign-ups before pre-booking opens

COLLECTION	FIELDS	USE
`meta`	`edition` {issued, version}, `serialSecret` {value}, `invoice:26-27` {seq}	Edition counter and lock, serial key, invoice counter per financial year

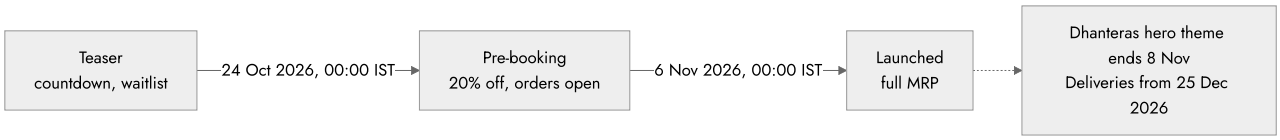
Indexes: users.email (unique), sessions.hash (unique), sessions.expiresAt (TTL), orders by userId + createdAt, status + holdUntil, coupon.code, waitlist.email (unique).

07 Key flows

Pre-order to certificate and invoice



Sales phases



08 Pricing & GST

MRPs include 18% GST. Discounts come off in order, and the GST is worked backwards from what the customer pays.

1. **Unit price:** MRP, or MRP × 0.8 during pre-booking (rounded to the rupee).
2. **Coupon:** a percentage (1–90%) or a fixed amount off that subtotal, never below ₹1.
3. **GST inside the total:** `gst = total - round(total ÷ 1.18)`.
4. **Invoice split:** buyer in Uttar Pradesh (seller's state, GSTIN 09AANCB7737M1ZV) gets CGST 9% + SGST 9%; any other state gets IGST 18%. HSN 9102.

Worked example: one of each, buyer in Uttar Pradesh, 5% coupon

MRP 49,999 + 39,999	₹89,998
Pre-booking privilege 20%	- ₹18,000
Subtotal (39,999 + 31,999)	₹71,998
Coupon 5%	- ₹3,600
Total charged	**₹68,398**
Taxable value · CGST 9% · SGST 9%	₹57,964 · ₹5,217 · ₹5,217

09 Concurrency & integrity

Never over-sell

Every reservation and payment confirmation runs in a transaction that writes one `meta/edition` document. Concurrent buyers conflict and retry one after another, so the 200 pieces and the last piece can't be double-sold.

Holds

A new order holds its pieces for 30 minutes. Paid orders and live holds count against stock, coupon limits and the wall; expired holds simply stop counting.

Edition numbers

Issued in payment order from `meta/edition.issued`. Serial = model code + number + 4-character HMAC check code, so serials can't be guessed.

Invoice numbers

Gap-free per financial year (April–March) from `meta/invoice:YY-YY`, assigned inside the payment transaction.

Idempotent payment

Verifying an already-paid order returns it unchanged; a payment that lands after the edition fills is refused before numbers are issued.

Server-side prices

The client sends only slugs, quantities and a coupon code. The server recomputes prices, caps 2 per model, and re-checks the coupon inside the transaction.

10 Security

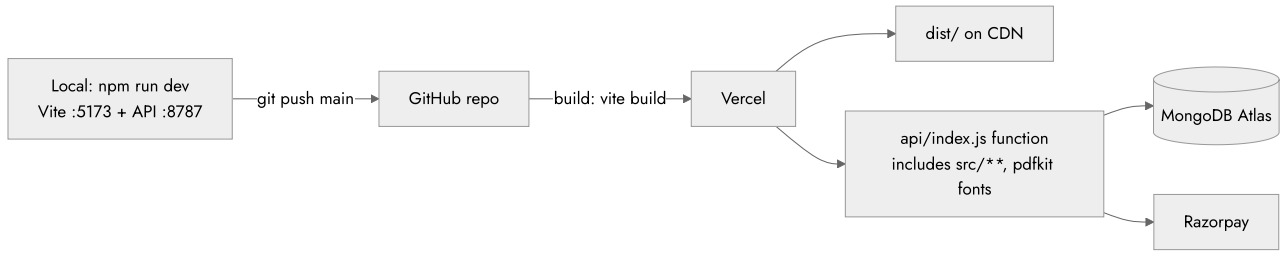
- **Passwords:** scrypt with a random salt; constant-time comparison.
- **Sessions:** 32-byte random tokens; only their SHA-256 is stored; httpOnly cookies (`SameSite=Lax` for customers, `Strict` for admin), `Secure` in production.
- **Admin:** username and password from environment variables, compared in constant time; 12-hour session; 5 failed sign-ins per 15 minutes.
- **Payments:** Razorpay signature verified with HMAC-SHA256 and the key secret; no card or UPI data touches the server.
- **Access:** customers see only their own orders and invoices; the public wall shows only opted-in display names.
- **Abuse limits:** login and register limits, coupon-check limit, CSV cells escaped against spreadsheet formula injection.
- **Secrets:** `.env` is git-ignored; production values live in Vercel environment variables.

11 Performance

- **Code splitting:** React, the motion library and the app ship as separate long-cached files; every page except home loads on demand.
- **Images:** 47 WebP files with 768 and 1152 px variants via `srcset`; below-the-fold images load lazily; the hero watches are preloaded.
- **Animation budget:** the WebGL silk renders a capped pixel count (30 fps on phones); particles stamp a pre-drawn sprite; gold veins render once on phones; countdowns re-render only their numbers; coins and banners animate with transforms only.

- **Caching:** fingerprinted assets cached for a year, images for 30 days, `index.html` always revalidated; gzip on the Node server.

12 Deployment & config



VARIABLE	PURPOSE
`MONGODB_URI`, `MONGODB_DB`	Atlas connection and database (`rubiik`)
`ADMIN_USERNAME`, `ADMIN_PASSWORD`	Admin dashboard sign-in
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Real payments; without them the store runs in test mode
`SERIAL_SECRET` (optional)	Key for serial check codes; otherwise generated once and kept in `meta`
`MONGODB_DNS` (optional)	Fallback DNS for the `mongodb+srv` lookup
`NODE_ENV=production`	Marks cookies `Secure`

Atlas Network Access must allow `0.0.0.0/0` for Vercel. Scripts: `npm run dev`, `npm run dev:open` (forces pre-booking open), `npm run build`, `npm start`, `npm run images`.

13 Admin dashboard

Overview

Pieces sold, held and left; revenue; customers; waitlist; sales per model; last 14 days; recent orders.

Orders

Search and filter, full details with certificates, coupon and invoice links, CSV export.

Customers

Accounts with orders, pieces, spend and serials.

Coupons

Create percentage or fixed codes with optional limit and expiry; pause, resume, delete unused; uses and discount given.

Waitlist · Owners wall

"Notify me" emails with CSV; show, hide or rename each paid order's wall card.

Media · Design system

Every image in use and the brand originals; live palette, type, logo, ornaments and the certificate.

14 Gaps & next steps

Ordered by impact on real customers.

Payment webhook

An order is confirmed only when the customer's browser calls `/verify`. If they close the tab right after paying, the money is taken but no certificate is issued. Add a Razorpay `payment.captured` webhook that runs the same confirmation.

Live keys and deployment

Vercel still serves an older build. Set the environment variables (Razorpay live keys, rotated Mongo password), push, then run one real payment end to end.

Emails

No order confirmation, invoice or launch emails yet; customers rely on their account page.

Password reset

Accounts have sign-in and sign-up only.

Invoice timing

A tax invoice is issued at payment although watches ship later; confirm with a CA whether advances need a receipt voucher instead, and the HSN code.

Rate limits per instance

Limits live in memory, so each serverless instance counts separately. A shared store (Mongo or Redis) would make them global.

Small items

Support hours are a placeholder; invoices print "Rs." instead of ₹; one shared admin account with no audit log.